

The Horizons Workflow: From Storage to Switch

Welcome to the architect's guide to Horizons. As you enter this environment, imagine yourself as a lead technician stepping up to a high-performance, industrial workbench. In traditional software, applications often fail by trying to "guess" your intent—silently loading heavy model weights in the background and crashing before you can even see the interface. Horizons rejects this. It is a stable, empty-at-boot workspace where **you** are the loader.

1. Foundations: The App as a Stable Workbench

The core philosophy of Horizons is stability through manual agency. We treat the Kotlin UI as a visual "Operating System" or workbench, while the "heavy hitters"—the Chromium browser, the Terminal, and the inference engines—act as independent **guest ports** running underneath it. By keeping the workbench empty at launch, we ensure the system remains responsive and lightweight, regardless of the heavy assets you intend to deploy. | Feature | Legacy "Auto-Boot" (Unstable) | Horizons Manual Loading (Stable) || ----- | ----- | ----- || **Startup State** | Attempts to auto-detect and load models secretly. | Boots to a clean, empty "workbench" state. || **Memory Usage** | High and unpredictable; prone to "OOM" kills. | Low and stable; resources used only upon activation. || **Crash Risk** | High risk of silent background failures. | Virtually zero; manual "fuses" prevent structural crashes. || **User Agency** | System-driven; "black box" logic. | **Architect-driven** ; manual plating of guest ports. |

This stable workbench is organized into a specific spatial geometry where each "room" serves a distinct stage of mechanical assembly.

2. The Seven-Tile Geometry: Mapping Your Command Center

Horizons is mapped across a seven-tile clock-face layout. To master the workflow, you must understand where your tools are stored and where the work is performed:

- **Horizons Panel:** The deck for system information, credits, and legal blueprints. [Future home of video game easter egg]
- **Monitor / Browser:** The dashboard for "window-shopping" your library, or the Internet, loading parameters & running "green-light" safety checks.
- **Chat:** The primary engagement interface; the agentic engine's front-end.
- **Router / Fuse Box (Center):** The final staging area where configurations are prepped, loaded and activated.
- **Settings / Vault:** The vault for raw ingredients—API keys, tokens, SDKs, and model weights.
- **Terminal / Mod Garage:** The space for scripting, bash commands, and defining custom runtimes.
- **Archives:** The storage bay for exported harnesses, saved environments, retired model pathways and historical artifacts. All stored as inert Files.
- Understanding this 7 tile layout is the first step toward moving a file from storage into a live, agentic engine.

3. Step 1: Landing, Loading, Calling or Creating

4. Step 2: Activation (The Router Fuse Box)

5. Step 3: Verification (The Monitor Checkpoint).

Note: ALL 6 TILES CAN ACCESS(push) TO THE ROUTER

CliffordService , a dedicated background daemon manager. We built CliffordService as a separate process because Android tends to "silently kill" background threads when a UI gets heavy; by keeping the daemon separate, your inference remains alive even if the interface is under load. When you are in the Router, you have three primary architectural controls:

- **The Re-Check on Flip:** When you flip the switch to "ON," the Router re-runs the green-light checks instantly. If you unplugged a model in the Armory, the switch won't close.
- **The "RUNNING" State:** This is the live handshake where the daemon is engaged and the guest port is open.
- **Sleep/Archive Options:** You can "Sleep" a config (unload it from memory but keep the "on the deck" settings) or long-term storage in the Archives.
- **The "Goat" Failure State:** If a handshake fails or a fuse blows during activation, the system triggers the `// GOAT_SAYS_NO` state. Rather than a silent crash, you are greeted with a structural failure banner and a synthesized, warbling "meh-eh-eh" bleat, alerting you that the machine needs a manual adjustment in the Mod Garage. This manual control creates a powerful, agentic workflow where you are the master of the machine's resources.

Conclusion: The Obsidian Vision

A successful workflow is a sensory experience. You move from an "empty cart" to a high-performance engine, watching the UI transform into a landscape of **volcanic glass facets** with **specular glints** . As you flip the switch, **layered glow nodes** pulse along the plasma conduits, and the **amber sun** of the Horizons icon illuminates the horizon. By following this journey— **Landing, Verification, and Activation** —you ensure your system remains a stable, powerful workbench. You are no longer a spectator to background processes; you are the lead architect of your own agentic engine.

1/3x1@